

dataQ 자동 표준화 단어 분리 알고리즘 전환 (greedy → DP)

시점: 2026-03-30 ~ 2026-04-02

이 일지의 주제

앞선 일지(03)의 끝에서 "selectCoveringTokens 개선 — OKT 토큰만 쓰지 말고 TB_WORD 기반 greedy longest-match를 도입한다"라는 과제가 남았다. 이 일지는 그 과제를 가지고 greedy로 한 번 구현했다가, 반례를 만나고, 국소 보정을 네 번 시도한 뒤, DP(동적 계획법) 기반 전역 최적 분리로 전환한 과정을 정리한다.

연구일지 중 분량이 가장 많은 편이고, 접근 방식의 방향 전환이 일어난 구간이다.

왜 분리 품질이 중요한가

자동 표준화 추천 파이프라인의 첫 단계가 "한글 용어명을 단어 단위로 분리"다. 분리가 정확해야 이후의 TB_WORD 매칭·영문약어 조합·도메인 추천이 정상적으로 이어진다. 분리가 어긋나면 이후 단계의 결과는 의미를 잃는다.

분리에는 OKT 형태소 분석기를 사용한다. 다만 OKT는 한국어 일반 텍스트용 분석기이지 "dataQ 표준단어 사전 기반 분리기"가 아니다. IT 도메인 약어·복합명사에서 기대와 다른 결과가 자주 나온다. 따라서 OKT 단독으로는 부족하고 TB_WORD·TB_WORD_DICT와 함께 사용해야 한다. "어떻게 조합할 것인가"가 이 일지의 주제다.

v0: OKT 토큰만 사용 (2026-03-27, 초기 단계)

최초 구현은 OKT 결과를 그대로 후보로 사용했다. 한계는 곧 드러났다. 테슬라코일제목이 [테슬라코일제, 목]으로 잘리고, 기준일자 는 한 덩어리로 나오며, "DB"나 "USR" 같은 약어는 의미 있는 단위로 인식되지 않았다. TB_WORD와의 연결 규칙도 아직 없었기 때문에 추천 결과를 그대로 쓸 수 없는 상태였고, 다음 단계로 넘어갔다.

v1: greedy longest-match (2026-03-30)

처음 정한 요구사항은 한 문장이었다.

"왼쪽부터 훑으면서, 그 위치에서 가장 길게 매칭되는 단어를 TB_WORD → TB_WORD_DICT → OKT 순으로 잡아라."

이 한 문장에 "우선순위"와 "탐색 전략"이 뭉뚱그려져 있다는 걸 그때는 몰랐다. 구현은 단순했다.

```
pos = 0
while pos < input.length:
    1) TB_WORD 중 input[pos..]로 시작하는 가장 긴 것
    2) 없으면 TB_WORD_DICT 중 가장 긴 것
    3) 없으면 OKT 후보 중 가장 긴 것(2글자 이상)
    4) 없으면 한 글자 스킵
    pos += matched.length
```

TB_WORD가 최우선이니 등록 단어는 언제나 잡힐 거라고 믿었다. 테스트도 "사용자이름", "운전면허번호", "주소" 같은 정상 케이스에 대해서는 깨끗하게 통과했다.

v1의 반례

반례는 곧 나왔다. 테슬라코일제목 이었다. TB_WORD에 제목 이 등록된 상태에서 돌려봤는데, 결과에 제목 이 없었다. 추적해 보니 이렇다.

```
pos=0에서 후보 탐색:
TB_WORD: "테슬라코일제목"으로 시작하는 등록 단어 없음
DICT:     없음
OKT:      "테슬라코일제"(6글자) ← 이것이 먼저 먹힘
pos=6: 남은 건 "목", 매칭 불가
결과: [테슬라코일제, 목] - 둘 다 UNRECOGNIZED
```

TB_WORD의 제목 은 사전에 있었다. 그런데 탐색의 시야에 들어오지 못했다. 이유는 이렇다. greedy는 왼쪽에서 결정을 내리면 되돌아오지 않는다. 앞의 OKT 토큰이 전역을 선점해버리면, 뒤에 있는 TB_WORD는 애초에 고려 대상이 되지 않는다. TB_WORD 우선순위라는 건 "같은 위치에서 동시에 보이는 후보들 사이의 우선순위"였을 뿐이다. 앞쪽 OKT와 뒷쪽 TB_WORD는 같은 위치에서 만나지 않는다. 깨닫고 나면 당연한 얘기지만, 당시에는 바로 알고리즘을 갈아엎지 않고 greedy를 살리는 방향으로 몇 번 더 시도했다. 아래가 그 기록이다.

국소 보정 네 번, 전부 실패

시도한 보정과 왜 실패했는지를 그대로 기록한다.

1. **OKT 5글자 이상 토큰을 버린다.** 테슬라코일제 (6글자)가 문제였으니 긴 OKT 토큰을 자르면 될 것 같았다. 돌려보니 로그인정보, 관리자계정 같은 정상적인 5글자 일반명사까지 같이 버려져서, 이 경우들은 오히려 미매칭이 증가했다. 반례 하나를 고치려고 여러 정상 케이스를 깨뜨렸다.
2. **OKT는 2글자 토큰만 허용.** 가장 보수적인 방향. 사용자이름 이 [사, 용, 자, 이, 림] 수준으로 쪼개지며 정상 케이스가 완전히 무너졌다. 즉시 되돌림.
3. **TB_WORD는 더 짧아도 OKT보다 무조건 먼저.** 이 방향으로 가면 "테슬라코일제목"은 앞에서 OKT 테슬라코일제 가 먹히는 대신 TB_WORD 무엇이 와야 한다. 그런데 한글명의 앞쪽에 맞는 짧은

TB_WORD가 있어도, 그게 전체 분리의 최적과는 전혀 별개다. 오히려 앞쪽에 짧은 TB_WORD가 있으면 그게 뒷쪽의 긴 TB_WORD 후보까지 선점해버리는 대칭적인 문제가 생긴다.

4. 후보 세트에 TB_WORD의 부분문자열을 전부 집어넣는다. 후보가 적어서 생긴 문제인가 싶어 후보를 잔뜩 늘렸다. 결과는 같았다. greedy는 어차피 현 위치의 최장만 보기 때문에, 후보 개수가 늘어나도 탐색 전략이 동일해서 선택이 달라지지 않는다.

네 번 다 "이 케이스는 고쳐지는데 저 케이스가 깨진다"는 패턴이었다. 그제야 문제가 점수 체계나 후보 구성에 있는 게 아니라 **탐색 전략 자체에 있다**는 걸 인정했다. greedy는 앞의 선택이 뒤 경로에 영향을 주지 않는다는 전제 위에서만 최적인데, 단어 분리는 그 전제가 명백히 깨진다. 앞에서 어떤 토큰을 고르느냐가 뒤에 남는 문자열 전체를 결정한다.

v2: DP 기반 최적 분리 (2026-04-02)

탐색 범위를 전역으로 바꾸기로 했다. 동적 계획법.

상태는 $dp[i]$ = 입력의 $0 \sim i$ 구간을 분리했을 때 얻을 수 있는 **최대 점수**. $parent[i]$ 와 $token[i]$ 는 역추적용. 전이는 각 위치 i 에서 모든 후보 토큰($OKT \cup TB_WORD \cup TB_WORD_DICT$)을 시도해서, $input[i..]$ 로 시작하는 것이 있으면 $dp[i + len]$ 을 갱신한다. 후보가 전혀 매칭되지 않을 때를 대비해 "한 글자 스킵" 전이도 남겼다. 페널티 -500.

점수 체계

층 사이 간격을 크게 벌리는 게 관건이었다. 짧은 TB_WORD 여러 개를 합친 경로가 긴 OKT 토큰 하나에 밀리지 않아야 하고, 반대로 한 글자 스킵이 정상 토큰을 대체하는 일도 없어야 한다.

토큰 유형	점수
TB_WORD	$10000 + \text{길이} \times 100$
TB_WORD_DICT	$5000 + \text{길이} \times 100$
OKT 2~4글자	$500 + \text{길이} \times 50$
OKT 5글자 이상	100
OKT 1글자	10
미매칭 1글자 스킵	-500

반례가 풀리는 경로

테슬라코일 제목 (길이 7)을 이 점수 체계 위에서 DP로 돌려보면 이렇게 된다.

후보: OKT: 테슬라코일제(6), 테슬라(3), 코일(2), 제목(2), 테(1)...

TB_WORD: 제목(2)

DICT: (해당 없음)

dp[0] = 0

위치 0에서:

"테슬라코일제"(OKT 6) → dp[6] = 100
"테슬라"(OKT 3) → dp[3] = 650
"테"(OKT 1) → dp[1] = 10

위치 3에서 (dp[3]=650, "테슬라" 뒤):

"코일"(OKT 2) → dp[5] = 1250
"코일제"(OKT 3) → dp[6] = 1300
"코일제목"(OKT 4) → dp[7] = 1350

위치 5에서 (dp[5]=1250, "테슬라"+"코일" 뒤):

"제목"(TB_WORD!) → dp[7] = 11450 ← 최고점
"제"(OKT 1) → dp[6] = 1260

dp[7] = 11450

역추적: 테슬라 + 코일 + 제목

앞쪽에서 테슬라코일제 (점수 100)를 선택해 dp[6]=100에 도달한 경로는, "테슬라(650) + 코일(600) + 제목(10200)" 합계 11450 경로와 같은 dp[7] 칸에서 직접 비교되어 자동으로 밀려난다. greedy 때는 이 비교 자체가 일어나지 않았다. 이게 전부다.

왜 greedy를 삭제하지 않았나

DP 구현에 버그가 있거나 후보 세트가 비정상적으로 텅 비어 dpSplit 이 빈 결과를 돌려주는 옛지 케이스에 대한 안전망으로 greedySplit 함수는 코드에 그대로 두었다. 현재 호출되지 않지만 회귀 방지용. 확신이 섰을 때 삭제할 예정이다.

같이 들어간 부대 개선

DP 교체와 같은 커밋에서 두 가지를 같이 넣었다.

- **2순위 분리 추천.** DP 결과는 "의미 단위 최대"로 보고, 그걸 한 번 더 잘게 쪼갠 결과를 2순위로 같이 보여준다. 사용자가 두 분리를 비교해서 고를 수 있다. "운전면허번호"를 [운전면허, 번호]로 줄 수도 있고 [운전, 면허, 번호]로도 줄 수 있는 식.
- **TB_WORD_DICT 시드 보강.** 테슬라, 코일, 데이터베이스, 로그인, 비밀번호, 이메일, 계좌, 거래, 잔액, 배송, 주문, 결제, 환불 같은 일반 IT/상거래 단어를 추가. 나중에 행안부 행정용어 / 국가기록원 기록관리 사전을 대량으로 얹으면서 DICTIONARY는 337건 → 48,159건까지 늘어난다.

그 다음날(2026-04-03)엔 dpSplit 자체의 버그 한 건("DICTIONARY 후보가 누락돼 단일 등록 단어 케이스에서 이상한 결과가 나오던 것")을 잡고, 표준화 추천의 한글명 실시간 DICTIONARY 조화·검증 강화·DICTIONARY 정리까지 한 번 더 돌렸다.

돌아보고 남긴 것

1. **알고리즘 이름보다 반례를 먼저 말한다.** 내가 처음 greedy라는 이름을 꺼낸 것 자체가 잘못은 아니었다. 잘못은 대표 반례를 같이 꺼내지 않은 것이다. "테슬라코일제목 같은 경우에 뒤쪽 TB_WORD가 잡혀야 한다"가 처음부터 같이 있었으면 greedy가 부적합하다는 게 설계 단계에서 드러났을 것이다.
2. **우선순위와 탐색 전략은 직교한다.** v1의 실패는 점수 체계가 틀린 게 아니라 비교 타이밍이 틀린 것이었다. TB_WORD를 최우선으로 둔다는 규칙이 있어도, 비교가 공정한 시점에 일어나지 않으면 규칙은 반영되지 않는다. "언제, 어떤 후보들 사이에서 비교가 일어나는가"를 먼저 그려야 점수 체계가 의미를 갖는다.
3. **국소 보정은 두 번이면 접는다.** "한 케이스 고치면 다른 게 깨진다"는 패턴이 두 번째 나오는 순간 전략 자체가 틀렸다는 신호다. v1에서 네 번을 돌렸는데, 두 번째에 이미 패턴이 보였고 그걸 세 번 더 끝었다. 다음부터는 두 번이면 접는다.
4. **OKT는 "정답 생성기"가 아니라 "후보 생성기"다.** OKT는 한국어 일반 텍스트에 학습된 분석기이므로, 표준사전 기반 분리에서 OKT의 역할은 탐색 공간을 넓히는 것이지 결정권을 갖는 것이 아니다. 최종 결정권은 점수 체계(TB_WORD 10000점)와 탐색 전략(DP)이 가져야 한다. v1에서는 OKT를 준-정답처럼 취급했기 때문에 greedy 결과에 그대로 반영됐다.

타임라인

- 2026-03-27 `da2b55b` 표준화 추천 신규 개발 (v0, OKT 단독)
- 2026-03-30 `5eb3e8e` 표준화 추천 고도화, greedy longest-match 도입 (v1)
- 2026-04-02 `7342c72` DP 설계서 작성
- 2026-04-02 `0f88184` DP 구현 교체, greedySplit은 fallback으로 보존 (v2)
- 2026-04-03 `14dc813` + dpSplit DICT 후보 누락 버그 수정, 검증 강화, DICT 정리 (v2.1)